Step 3 — Retries, Then a Safe Fallback, When a Model Call Fails — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: real API calls time out, hit rate limits, and sometimes just fail outright. The retry part is already there for free: the OpenAI client underlying `ChatOpenAI` retries timeouts, connection errors, 429s and 5xx errors **2 times with backoff** on its own (Doc04's error topic). Adding `.with_retry()` on top of that would stack a second, redundant retry layer — the exact mistake Doc02, Doc04 and Doc05 all warn against. What's actually missing is the part **after** the retries run out: today, the ticket just crashes. This step gives every model call a safe fallback chain to fall back to instead. **If not:** one bad minute at OpenAI empties your whole queue into tracebacks.

After this step the project runs as before, and when the model can't be reached, the ticket gets a safe canned reply instead of a crash. Files not shown here don't change.

Changed first

Changed from Step 2: `main.py` no longer catches `RateLimitError` — the fallback now handles it inside the pipeline — and it prints whether the fallback was used. A bad key is still caught: it's not a temporary problem, and no fallback should hide it.

Changed from Step 2: `TicketRecord` gets its first new field, `used_fallback`, with a default.

Story — `schemas.py`: this adds a field to `TicketRecord` recording whether the safe canned reply was sent instead of a real one. Giving it a default of `False` means the Step 1 test that builds a `TicketRecord` without setting this field still works unchanged. **If not:** a fallback reply would look exactly like a real one in your data, with no way to tell them apart later.

```
# src/schemas.py
# The pipeline's checked shapes. TicketRecord now also records whether the
# canned fallback reply was used; the default keeps earlier code working.
from typing import Literal

from pydantic import BaseModel

class TicketCategory(BaseModel):
    category: Literal["billing", "technical", "general"]
    reason: str
```

```
    sure: bool

class TicketRecord(BaseModel):
    customer_id: str
    ticket: str
    category: str
    reply: str
    used_fallback: bool = False     # new in Step 3
```

Story — `main.py`: with the fallback absorbing transient failures, only the errors a person must actually fix (like a bad key) still reach this loop; `print_record()` now shows one more line reporting whether the fallback fired. **If not:** a transient rate limit would still skip the ticket entirely instead of answering it with a safe canned reply.

```
# main.py
# The file you run: python main.py
# Loads the config, reads the sample tickets, runs handle_ticket() on each,
# and prints each result with print_record().
import json
import sys

import openai

from src.config import load_config
from src.exceptions import MissingConfigError
from src.services.pipeline import handle_ticket
from src.utils.logger import get_logger

def load_samples():
    with open("data/sample/tickets.json") as samples_file:
        return json.loads(samples_file.read())

def print_record(record):
    print(f"  category:      {record.category}")
    print(f"  used fallback: {record.used_fallback}")     # new in Step 3

def main():
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    samples = load_samples()
    print(f"LangChainPro — {len(samples)} tickets in the queue.")
```

```
    for sample in samples:
        print()
        print(f"Ticket from {sample['customer_id']}: {sample['ticket']}")
        try:
            record = handle_ticket(sample["ticket"], sample["customer_id"])
        # changed in Step 3: no RateLimitError here any more
        except openai.AuthenticationError:
            logger.error("Authentication failed — check OPENAI_API_KEY.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        print_record(record)

if __name__ == "__main__":
    main()
```

Back to this step · Hint 1 · Hint 2 · Solution

Solution — `.with_fallbacks()` on each chain, only for transient errors

Story — `chains/fallback.py`: this holds both backups in one file. `TRANSIENT_ERRORS` is a tuple listing only the failures that genuinely resolve on their own (timeouts, connection errors, rate limits) — it's passed to `exceptions_to_handle` so the fallback catches only those, never a real bug. `with_canned_reply()` and `with_unsure_category()` each wrap a chain with `.with_fallbacks([...], exceptions_to_handle=TRANSIENT_ERRORS)` (Doc05's LCEL pattern); `RunnableLambda` turns a plain Python function into a step `.with_fallbacks()` can accept as its backup. The design decision worth naming: a canned reply is fine even for a double charge, because it promises nothing specific, and every fallback use gets flagged for a human (Step 4). **If not:** a bug in your own code (say, a `KeyError`) would be silently hidden behind a polite canned reply, with `exceptions_to_handle` making sure that can't happen.

```
# src/chains/fallback.py
# Safe backups for transient model failures only. TRANSIENT_ERRORS are the
# failures that pass on their own; with_canned_reply/with_unsure_category
# wrap a chain so a bad key or a code bug still fails loudly.
import openai
from langchain_core.runnables import RunnableLambda

from src.schemas import TicketCategory
from src.utils.logger import get_logger

TRANSIENT_ERRORS = (
    openai.APIConnectionError,     # APITimeoutError is a kind of this
    openai.RateLimitError,
    openai.InternalServerError,
)

FALLBACK_REPLY = (
    "Thanks for contacting SupportCo. We've received your message, "
    "and a member of our team will reply within one business day."
)

def canned_reply(inputs):
    logger = get_logger(__name__)
    logger.warning("Reply chain failed - sending the fallback reply")
    return FALLBACK_REPLY

def unsure_category(inputs):
```

```
    logger = get_logger(__name__)
    logger.warning("Classifier failed - using general, marked unsure")
    return TicketCategory(
        category="general",
        reason="The classifier was unavailable.",
        sure=False,
    )

def with_canned_reply(chain):
    backup = RunnableLambda(canned_reply)
    return chain.with_fallbacks(
        [backup], exceptions_to_handle=TRANSIENT_ERRORS
    )

def with_unsure_category(chain):
    backup = RunnableLambda(unsure_category)
    return chain.with_fallbacks(
        [backup], exceptions_to_handle=TRANSIENT_ERRORS
    )
```

Changed from Step 2: the classifier chain is wrapped with its backup.

Story — `chains/classifier.py`: wrapped with `with_unsure_category()`, if the classifier itself can't be reached, the ticket falls back to "general, not sure" — and still gets a reply instead of stalling. **If not:** a classifier outage would stop every ticket before any reply chain even ran, even though the reply chains themselves might be perfectly fine.

```
# src/chains/classifier.py
# The classifier chain, now wrapped so a model outage falls back to
# "general, not sure" instead of crashing the ticket.
from langchain_core.prompts import ChatPromptTemplate
from pydantic import ValidationError

from src.chains.fallback import with_unsure_category     # new in Step 3
from src.models.llm import get_model
from src.schemas import TicketCategory
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def build_classifier_chain():
    prompt = ChatPromptTemplate.from_template(load_prompt("classifier"))
    model = get_model().with_structured_output(TicketCategory)
    chain = prompt | model
    return with_unsure_category(chain)     # changed in Step 3: wrapped

def classify_ticket(ticket):
```

```
    logger = get_logger(__name__)
    chain = build_classifier_chain()
    try:
        result = chain.invoke({"ticket": ticket})
    except ValidationError:
        logger.warning("Classifier answer did not fit TicketCategory")
        result = TicketCategory(
            category="general",
            reason="The classifier's answer could not be read.",
            sure=False,
        )
    logger.info(
        f"category={result.category} sure={result.sure} "
        f"reason={result.reason}"
    )
    return result
```

Changed from Step 2: every reply chain is wrapped with the canned reply.

Story — `chains/replies.py`: the `with_canned_reply()` wrapper goes around the **whole** reply chain, lookups included, so a transient failure anywhere inside it — not just the final model call — triggers the same fallback. **If not:** you'd need to wrap the lookup step and the reply step separately, three times over for three categories.

```
# src/chains/replies.py
# build_reply_chain(category): the per-category reply chain, now wrapped so
# a transient model failure sends the canned reply instead of crashing.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.chains.fallback import with_canned_reply     # new in Step 3
from src.chains.lookups import build_lookup_step
from src.models.llm import get_model
from src.utils.prompts import load_prompt

def build_reply_chain(category):
    prompt = ChatPromptTemplate.from_template(load_prompt(category))
    chain = prompt | get_model() | StrOutputParser()

    if category == "general":
        full_chain = chain
    else:
        full_chain = build_lookup_step() | chain

    return with_canned_reply(full_chain)     # changed in Step 3: wrapped
```

Changed from Step 2: the record says whether the fallback ran.

Story — `services/pipeline.py`: the canned reply is always the exact same fixed text, so a simple string comparison against `FALLBACK_REPLY` is all `handle_ticket()` needs to tell whether it was used, and set the new flag accordingly. **If not:** a fallback would be invisible in your saved data — Step 4's human-review flag needs this to exist.

```
# src/services/pipeline.py
# handle_ticket(): classify, reply, build the record — and now record
# whether the canned fallback reply was used.
from src.chains.classifier import classify_ticket
from src.chains.fallback import FALLBACK_REPLY     # new in Step 3
from src.chains.replies import build_reply_chain
from src.schemas import TicketRecord

def handle_ticket(ticket, customer_id):
    category = classify_ticket(ticket)
    chain = build_reply_chain(category.category)
    inputs = {"ticket": ticket, "customer_id": customer_id}
    reply = chain.invoke(inputs)
    print(f"Reply: {reply}")

    # new in Step 3: the fallback always returns exactly FALLBACK_REPLY
    used_fallback = reply == FALLBACK_REPLY

    return TicketRecord(
        customer_id=customer_id,
        ticket=ticket,
        category=category.category,
        reply=reply,
        used_fallback=used_fallback,     # new in Step 3
    )
```

Story — `tests/unit/test_fallback.py`: a fallback you can't prove fires — and can't prove stays quiet when nothing's wrong — isn't really tested at all. Case 1 points a real `ChatOpenAI` at a port where nothing is listening, forcing a genuine connection failure with no API key and no cost. Case 2 proves a healthy, working chain never touches the fallback unnecessarily. Case 3 proves a real bug in your own code is **not** silently swallowed by it. **If not:** you'd find out the fallback doesn't actually work during a real production outage.

```
# tests/unit/test_fallback.py
# Proves the fallback runs when the model can't be reached, and only then.
# Each case prints OK or WRONG. No API key needed.
# Run from the repo root: python -m tests.unit.test_fallback
from langchain_core.output_parsers import StrOutputParser
```

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableLambda
from langchain_openai import ChatOpenAI

from src.chains.fallback import FALLBACK_REPLY, with_canned_reply

def build_broken_chain():
    # nothing listens on port 9, so every call fails at once; max_retries=0
    # skips the client's 2 retries, so it's instant
    model = ChatOpenAI(
        model="gpt-4o-mini",
        api_key="sk-test",
        base_url="http://127.0.0.1:9",
        max_retries=0,
    )
    prompt = ChatPromptTemplate.from_template("Reply to: {ticket}")
    return prompt | model | StrOutputParser()

def working_step(inputs):
    return "A real reply."

def buggy_step(inputs):
    return inputs["no_such_key"]

def test_outage_uses_fallback():
    chain = with_canned_reply(build_broken_chain())
    reply = chain.invoke({"ticket": "Where is my invoice?"})
    ok = "OK"
    if reply != FALLBACK_REPLY:
        ok = "WRONG"
    print(f"case 1 (outage falls back) -> {ok}")

def test_working_chain_skips_fallback():
    chain = with_canned_reply(RunnableLambda(working_step))
    reply = chain.invoke({"ticket": "Hi"})
    ok = "OK"
    if reply != "A real reply.":
        ok = "WRONG"
    print(f"case 2 (working, no fallback) -> {ok}")

def test_bug_is_not_hidden():
    chain = with_canned_reply(RunnableLambda(buggy_step))
    ok = "WRONG"
    try:
        chain.invoke({"ticket": "Hi"})
    except KeyError:
        ok = "OK"
```

```
    print(f"case 3 (bug still raises) -> {ok}")

if __name__ == "__main__":
    test_outage_uses_fallback()
    test_working_chain_skips_fallback()
    test_bug_is_not_hidden()
```

Run it (from the repo root):

```
python -m tests.unit.test_fallback
python main.py
```

`test_fallback` prints (after one warning log line):

```
case 1 (outage falls back) -> OK
case 2 (working, no fallback) -> OK
case 3 (bug still raises) -> OK
```